

UNIVERZITET U BEOGRADU
MATEMATIČKI FAKULTET



Vukan Antić

UPOREĐIVANJE PROTOKOLA ZA
KOMUNIKACIJU GRAPHQL, GRPC I REST,
RAZVOJOM MOBILNE APLIKACIJE
UPOTREBOM TEHNOLOGIJA SPRING BOOT
I REACT NATIVE

master rad

Beograd, 2026.

Mentor:

dr Vladimir FILIPOVIĆ, redovan profesor
Univerzitet u Beogradu, Matematički fakultet

Članovi komisije:

dr Aleksandar KARTELJ, vandredni profesor
Univerzitet u Beogradu, Matematički fakultet

dr Staša VUJIČIĆ STANKOVIĆ, docent
Univerzitet u Beogradu, Matematički fakultet

Datum odbrane: _____

*Ovaj rad posvećujem svom ocu, majci, bratu,
prijateljima i kolegama. dr. Vladimiru Filipoviću se
zahvaljujem na strpljenju kao i savetima u toku izrade
ovog rada. Posebnu zahvalnost dugujem partneru Dušici,
na svoj ljubavi pruženoj u toku studija.*

Naslov master rada: Upoređivanje protokola za komunikaciju GraphQL, gRPC i Rest, razvojem mobilne aplikacije upotrebom tehnologija Spring Boot i React Native

Rezime: Sa porastom broja mobilnih aplikacija, komunikacija između klijenta i servera najčešće se svodi na upotrebu arhitektonskog stila REST. Pored njega postoje protokoli GraphQL i gRPC, koji donose drugačije ustupke po pitanju fleksibilnosti, veličine poruka i dogovora između dve strane.

U ovom radu razvijena je mikroservisna aplikacija za svakodnevni prikaz umetničkih dela, upotrebom tehnologija Spring Boot i React Native. Prikazan je postupak prikupljanja i obrade podataka iz javno dostupne arhive Instituta za umetnost u Čikagu, podela serverske strane na mikroservise, organizacija svakog servisa po onion arhitekturi, kao i klijentska aplikacija zasnovana na obrascu model-prikaz-kontroler. Nad tako izgrađenim sistemom, implementirana su sva tri protokola, pri čemu su ostali slojevi aplikacije ostali nepromenjeni, čime je omogućeno njihovo neposredno poređenje. Za svaki protokol prikazani su osnovna zamisao, implementacija na serveru i na klijentu, kao i prednosti i nedostaci uočeni pri radu. Razvijena aplikacija dostupna je kao softver otvorenog koda.

Ključne reči: protokoli komunikacije, REST, GraphQL, gRPC, onion arhitektura, React Native, Spring Boot

Sadržaj

1	Uvod	1
2	Prikupljanje podataka	3
2.1	Obrada podataka	4
2.1.1	Filtriranje	5
2.1.2	Provera ispravnosti slike	6
2.1.3	Relacioni model	6
2.1.4	Upisivanje u bazu	8
3	Serverska strana	10
3.1	Spring Boot	10
3.2	Podela mikroservisa	10
3.3	Onion arhitektura unutar servisa	12
3.3.1	Jezgro: entiteti i objekti za prenos	14
3.3.2	Srednji sloj: poslovna logika	15
3.3.3	Spoljašnji sloj: adapteri protokola	17
3.4	Asinhrona komunikacija	18
4	Klijentska aplikacija	20
4.1	React Native	20
4.2	Model-Prikaz-Kontroler arhitektura	20
4.2.1	Prikaz	21
4.2.2	Model	21
4.2.3	Kontroler	22
4.3	Namera	23
4.4	Repozitorijum	24
4.5	Rukovalac komandama	25

4.6	Videti	27
5	Protokoli komunikacije	29
5.1	Računanje vremena utrošenog na čekanje	29
5.2	REST	30
5.2.1	Arhitektonska ograničenja	30
5.2.2	Nedostaci	31
5.2.3	Implementacija	31
5.3	GraphQL	32
5.3.1	Schema	33
5.3.2	Upiti, mutacije i pretplate	34
5.3.3	$N + 1$ problem	35
5.3.4	Keširanje i cena upita	35
5.3.5	Implementacija	35
5.4	gRPC	37
5.4.1	Protocol Buffers	37
5.4.2	HTTP/2 i vrste poziva	38
5.4.3	Nedostaci	40
5.4.4	Implementacija	40
6	Zaključak	42
	Bibliografija	44

Glava 1

Uvod

Sa rastućom upotrebom mobilnih aplikacija komunikacija između klijenta i servera, u većini slučajeva, svodi se na Rest [1] protokol komunikacije, iako postoje i drugi protokoli poput gRPC [2] i GraphQL [3], koji su performantniji, fleksibilniji kao i skalabilniji. Zato su u toku izrade naredne aplikacije sva 3 protokola bila upotrebljena, da bi se moglo videti kada je koji protokol najbolje iskoristiti.

Sam projekat predstavlja aplikaciju otvorenog koda¹, koja svakodnevno daje korisniku umetničku sliku, sa detaljnim opisom, i mogućnost da sačuva omiljenu, radi daljeg preporučivanja novog sadržaja. Pored toga, postoje i funkcionalnosti čuvanja omiljenih slika u memoriji sopstvenog telefona, kao i deljenje slika sa drugim korisnicima. Takođe, pri izlasku nove slike, korisnik dobija obaveštenje o novom sadržaju koji je dodat za njega.

Izradu projekta čine sledeći delovi:

- Prikupljanje podataka - U glavi 2 predstavljen je detaljan opis odakle je aplikacija dobila podatke za prikaz u okviru same aplikacije.
- Izrada servera zasnovanog na mikroservisnoj arhitekturi [4] - U glavi 3 predstavljena je implementacija servera, koja je zasnovana na mikroservisnoj arhitekturi, dok je svaki od mikroservisa pratio (eng. *Onion*) arhitekturu [5, 6], koja je pomogla da se lako dodaje podrška za sve protokole komunikacije.
- Izrada klijenta zasnovanog na MVC arhitekturi [7] - U glavi 4 prikazano je kako je implementiran klijentski deo koda koji prati MVC arhitekturu i dodatne funkcionalnosti koje klijent podržava kao što je videt (eng. *widget*).

¹<https://github.com/VukanAntic/ArtOfTheDay>

- Dodavanje gRPC i GraphQL - Glava 5 predstavlja prikaz svih gore navedenih protokola komunikacije, kao i poređenja.

Glava 2

Prikupljanje podataka

Aplikacija razvijena u okviru ovog rada, *INSPIRA daily*, prikazuje korisniku novu umetničku sliku svakog dana uzimajući u obzir preference korisnika. Potrebe projekta su bile: velika količina umetničkih slika, detaljan opis za svaku, žanr kom slika pripada, kao i informacija ko je bio autor iste. Iz tih razloga, za izvor podataka bio je izabran Institut za umetnost u Čikagu (eng. *The Art Institute of Chicago*) [8]. Institut nudi javni programski interfejs, ali i potpuni prepis zbirke (eng. *data dump*) - arhiva svih slika, sačuvanih u JSON obliku. Odabran je pristup preko arhive, iz razloga što bi dohvaćanje podataka preko interfejsa zahtevalo veliki vremenski period, kao i stavljanje velikog pritiska na server instituta.

Slike se ne čuvaju lokalno, jer ih institut izlaže preko protokola IIIF (eng. *International Image Interoperability Framework*) koji omogućava da se željena slika, kao i njena širina zatraži preko adrese (*url*). Adresa slike se može sastaviti iz identifikatora slike (eng. *image id*) , što pokazuje Primer koda 2.1.

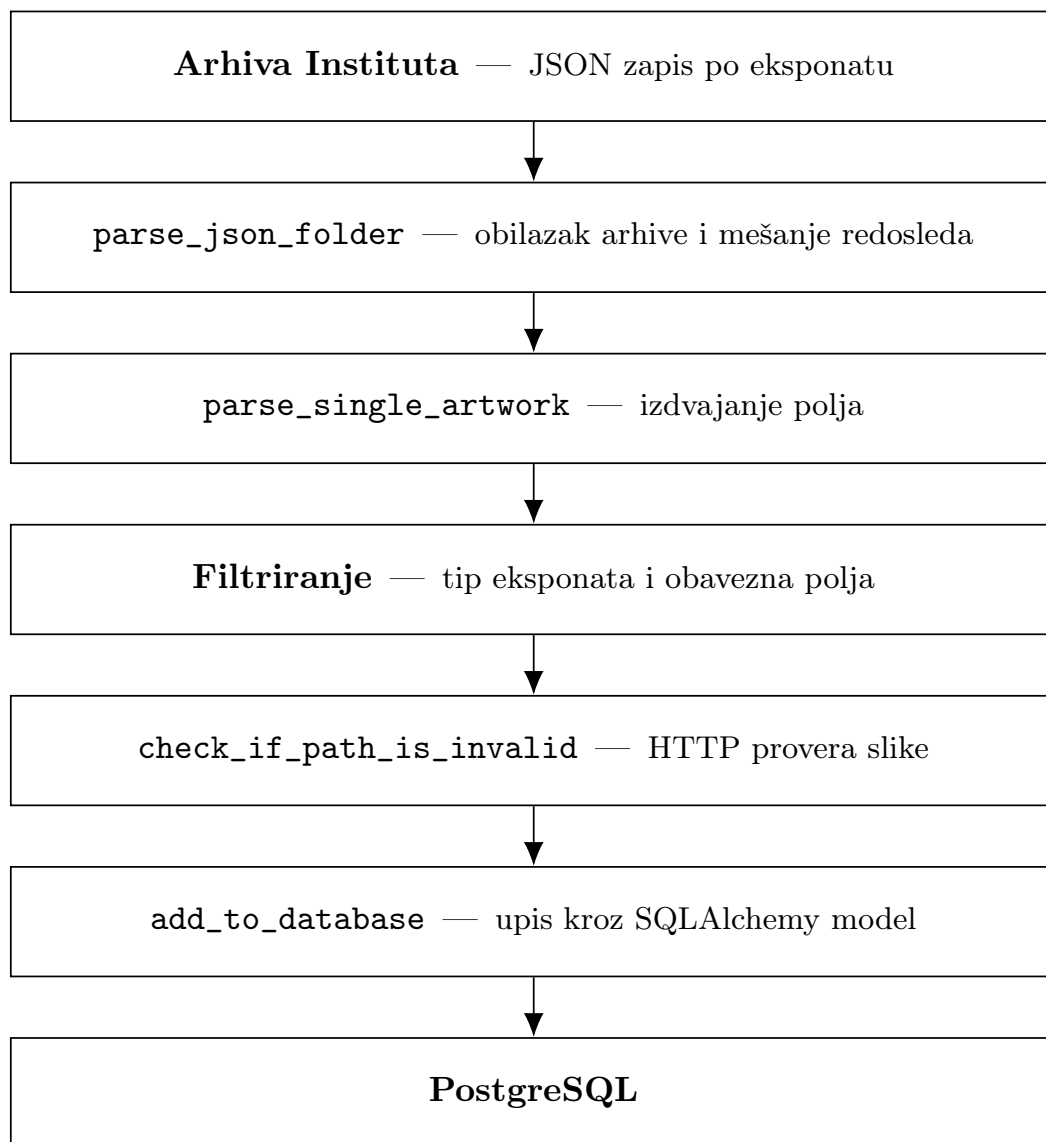
```
1 path = f'https://www.artic.edu/iiif/2/{data["image_id"]}'  
2      /full/843,/0/default.jpg'
```

Primer koda 2.1: Sastavljanje adrese slike iz identifikatora

Na ovaj način, za dohvaćanje i prikazivanje slika bilo je samo potrebno izvući identifikator svake željene slike, i dobija se adresa iste. Vrednost 843 podrazumeva najčešću korišćenu veličinu slike, koja je verovatno keširana na strani institutovog servera. Bilo je moguće zameniti tu vrednost sa manjom vrednošću, da sve slike imaju istu širinu i visinu.

2.1 Obrada podataka

Automatska obrada podataka bila je realizovana uz pomoć skripte napisane u programskom jeziku *Python* po imenu *ImageDatabaseGenerator.py*. Slika 2.1 prikazuje kako je to urađeno.



Slika 2.1: Tok obrade podataka

2.1.1 Filtriranje

Arhiva instituta ne sadrži samo umetničke slike, već i skulpture, fotografije, tekstil kao i veliku količinu drugih tipova objekata. Iz tog razloga, bilo je potrebno filtrirati podatke, tako da se pronađu samo umetničke slike, što se može videti u Primer koda 2.2.

```
1 def parse_single_artwork(data):
2     id = data["id"]
3     title = data["title"]
4     description = data["description"]
5     date_display = data["date_display"]
6     style_id = data["style_id"]
7     style_title = data["style_title"]
8     artist_id = data["artist_id"]
9     artist_title = data["artist_title"]
10    artwork_type_id = data["artwork_type_id"]
11    # id = 1 represents artworks
12    artworked_wanted_indicies = [1]
13    path = f'https://www.artic.edu/iiif/2/{data["image_id"]}/full
14           /843,/0/default.jpg'
15    if title is None or description is None or date_display is None
16       \
17    or style_title is None or artwork_type_id not in
18       artworked_wanted_indicies or \
19    check_if_path_is_invalid(path) :
20        return None
21    return {
22        "id" : id,
23        "title" : title,
24        "description" : description,
25        "date_display" : date_display,
26        "path" : path,
27        "style_title" : style_title,
28        "style_id" : style_id,
29        "artist_title" : artist_title,
30        "artist_id" : artist_id
31    }
```

Primer koda 2.2: Filtriranje umetničkih slika

Pored provere da li je u pitanju umetnička slika, bilo je potrebno dodati i druge

provere da li postoji naziv slike, opis, i datum njenog nastanka.

2.1.2 Provera ispravnosti slike

Naredni korak posle filtriranja podrazumeva proveru da li je slika koju imaju u arhivi instituta ispravna, tj. da li pristup adresi te slike vraća grešku, što pokazuje primer koda 2.3.

```
1 def check_if_path_is_invalid(path) :
2     response = requests.get(path)
3     print(path)
4     print(response.status_code)
5
6     if response.status_code == 429:
7         print("Too many requests in a short period of time, need to
            cool off...")
8         time.sleep(60)
9         response = requests.get(path)
10    if response.status_code == 200:
11        return False
12
13    print("Found an error code!")
14    return True
```

Primer koda 2.3: Provera dostupnosti

Bitno je napomenuti da programski interfejs koji je dostupan od instituta vraća grešku **429**, ukoliko je previše zahteva stiglo sa iste *IP adrese* u kratkom vremenskom periodu. Zbog toga se u toku izvršavanja skripte program zaustavi na 60 sekundi, da bi moglo dalje da se pristupi serveru instituta.

2.1.3 Relacioni model

Izdvojeni podaci upisuju se u bazu (eng. *PostgreSQL*) preko biblioteke (eng. *SQLAlchemy*). Potrebni podaci se čuvaju u tri različite tabele - Umetnička slika (eng. *Artwork*), Žanr (eng. *Genre*) kao i Umetnik (eng. *Artist*). To se može videti primerom koda 2.4, koji takav model generiše:

```
1 artwork_genre_association = Table(
```

```
2     'artwork_genre', Base.metadata,
3     Column('artwork_id', Integer, ForeignKey('artworks.id')),
4     Column('genre_id', String(20), ForeignKey('genres.id'))
5 )
6
7 class Artwork(Base):
8     __tablename__ = 'artworks'
9     id = Column(Integer, primary_key=True, autoincrement=True)
10    title = Column(String(255), nullable=False)
11    date_display = Column(String(255), nullable=False)
12    description = Column(Text, nullable=False)
13    path_to_artwork = Column(String(255), nullable=False)
14    artist_id = Column(Integer, ForeignKey('artists.id'), nullable=
        False)
15
16    # Relationships
17    genres = relationship('Genre', secondary=
        artwork_genre_association, back_populates='artworks')
18    artist = relationship('Artist', back_populates='artworks')
19
20 class Artist(Base):
21     __tablename__ = 'artists'
22
23     id = Column(Integer, primary_key=True, autoincrement=True)
24     # Unknown artist!
25     name = Column(String(255), nullable=True)
26
27     artworks = relationship('Artwork', back_populates='artist')
28
29 class Genre(Base):
30     __tablename__ = 'genres'
31
32     id = Column(String(20), primary_key=True)
33     name = Column(String(255), nullable=False)
34
35     # Define the relationship to Artwork
36     artworks = relationship('Artwork', secondary=
        artwork_genre_association, back_populates='genres')
```

Primer koda 2.4: Relacioni model

Tri bitne stavke o samom modelu koje je značajno izdvojiti.

- **Identifikatori se preuzimaju od izvora** - Delo i autor zadržavaju celobrojni identifikator koji im je dodelio institut, umesto da dobiju novi. Time je omogućeno da se u budućnosti zbirka dopuni novim slikama i umetnicima, bez potrebe za brigom o duplikatima. Ista odluka primenjena je i na žanrove, identifikator je zadržan od instituta, koji je niska.
- **Veza slika-žanr je više-prema-više** - Zbog prirode skupa podataka, jedna slika može pripadati više različitih žanrova u isto vreme, isto tako, jednom žanru može odgovarati veći broj slika.
- **Veza slika-umetnik je jedan-prema-više** - Isto kao i za prethodni odnos, zbog prirode podataka, jednu sliku je mogao da naslika jedan umetnik dok je jedan umetnik mogao da naslika više različitih slika.

2.1.4 Upisivanje u bazu

Nakon što se uspešno kreira model, prelazi se na naredni korak. Upisivanje podataka koji su uzeti iz arhive u novo kreiranu bazu. U dole navedenom kodu 2.5 možemo videti da, nakon što su entiteti kreirani i dodati u liste, u toku jedne izmene (eng. *commit*), podaci su dodati u bazu podataka.

```
1         if artist_id not in artist_ids:
2             artists_data.append(artist_data)
3         if artwork["style_id"] not in genre_ids:
4             genres_data.append(genre_data)
5
6         new_artist_data = list(filter(lambda elem : elem.id ==
7                                     artist_id, artists_data))
8         new_genre_data = list(filter(lambda elem : elem.id ==
9                                     artwork["style_id"], genres_data))
10        print(new_artist_data)
11        artwork_data.artist = new_artist_data[0]
12        artwork_data.genres.append(new_genre_data[0])
13
14        artwork_ids.add(artwork["id"])
15        artist_ids.add(artist_id)
16        genre_ids.add(artwork["style_id"])
17
18        session.add_all(artists_data)
```

```
18     session.add_all(genres_data)
19     session.add_all(artworks_data)
20     session.commit()
```

Primer koda 2.5: Isečak koda koji upisuje podatke u bazu podataka

Glava 3

Serverska strana

Serverska strana realizovana je u tehnologiji (eng. *Spring Boot*) kao skup od četiri međusobno nezavisna mikroservisa. Svaki od mikroservisa u sebi je implementiran preko (eng. *Onion*) arhitekture. Mikroservisi su međusobno povezani preko brokera poruka (eng. *broker software*) po imenu (eng. *RabbitMQ*). U narednoj glavi biće prikazano kako su organizovani svi mikroservisi, kako izgleda svaki pojedinačan mikroservis, kako komuniciraju i kako arhitektura olakšava dodavanje više protokola komunikacije da budu implementirani unutar istog mikroservisa, sa minimalnom količinom dodatnog koda.

3.1 Spring Boot

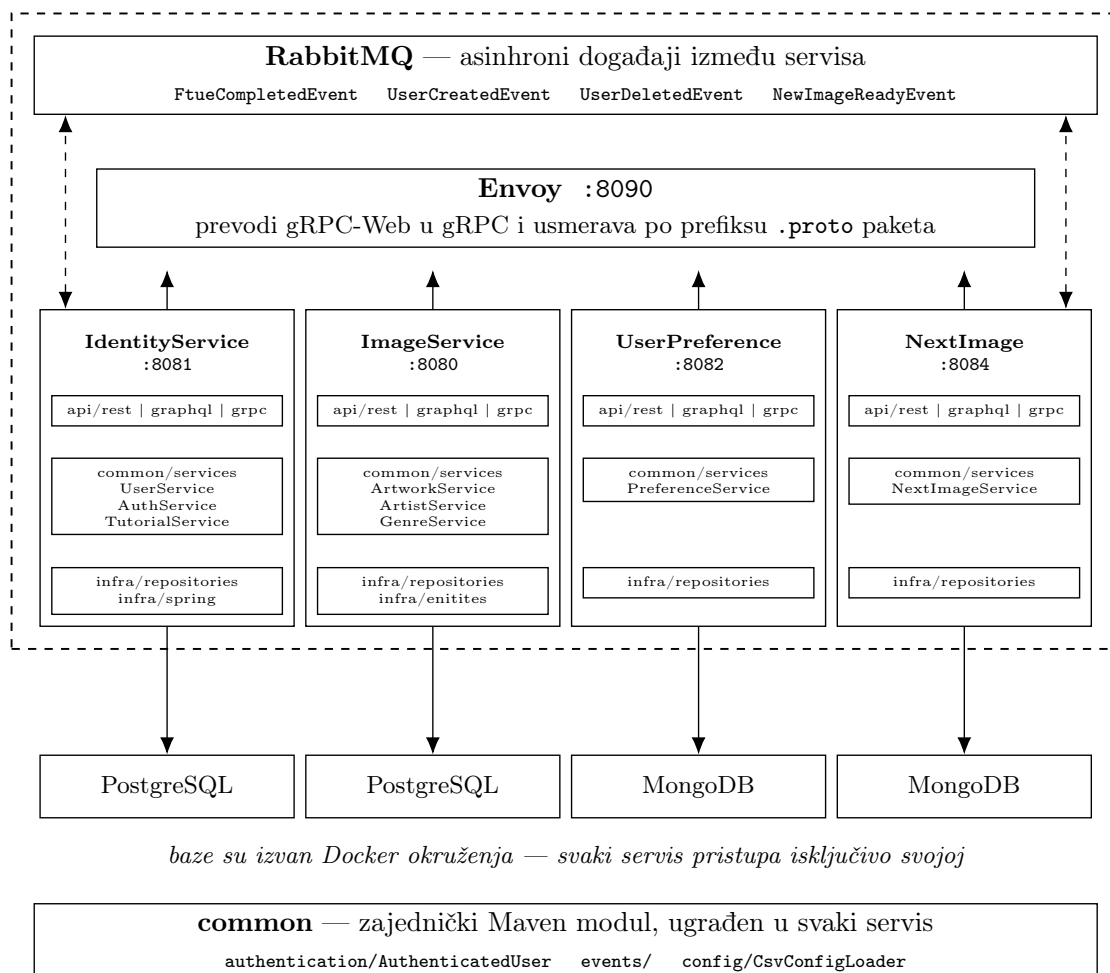
Čitav server razvijen je u programskom jeziku Java, uz radni okvir (eng. *framework*) *Spring Boot* [9]. Spring Boot je nadogradnja nad radnim okvirom *Spring*, uz motivaciju da ubrza razvoj, smanji konfiguraciju i ubrza puštanja u produkciju (eng. *deployment*). Pored toga, Spring Boot podržava svaki od protokola komunikacije koje ovaj rad opisuje. Zbog toga, server je izrađen u njemu.

3.2 Podela mikroservisa

Mikroservisi su međusobno nezavisne celine koje su najčešće podeljene po nekoj poslovnoj logici. Svaki od mikroservisa unutar sebe prati neku dogovorenu strukturu, ima sopstvenu bazu podataka, i tako omogućava da svaki od mikroservisa može da se pusti u produkciju sa njegovim najnovijim izmenama, bez pravljenja novog izdanja za ostale mikroservise. Iz tih razloga, odlučeno je da serverska stra-

na bude podeljena na mikroservise. Organizaciju čitavog servera možemo videti na slici 3.1.

Docker Compose — šest kontejnera



Slika 3.1: Arhitektura serverske strane

Kao što se vidi, serverska strana je podeljena na četiri mikroservisa:

- **Mikroservis za slike (eng. *ImageService*)** - Mikroservis odgovoran za sav sadržaj vezan za slike. On koristi podatke koji su iz Glave 2, i daje programski interfejs vezan za slike, umetnike i žanrove. Podaci se čuvaju u relacionoj bazi podataka *PostgreSQL*, jer su svi podaci međusobno povezani.
- **Mikroservis za autentifikaciju (eng. *IdentityService*)** - Mikroservis koji se bavi čitavim procesom autentifikacije korisnika. Unutar sebe sadrži programski interfejs za prijavu, odjavu, registraciju, kao i čuvanje vitalnih

informacija o korisniku, kao što su korisničko ime, lozinka i adresa elektronske pošte. Pri uspešnoj prijavi, servis izdaje par žetona: *pristupni žeton* kratkog trajanja (kojim se potpisuje svaki naredni zahtev) i *žeton za osvežavanje* dužeg trajanja (kojim se pribavlja nov pristupni žeton bez ponovnog unosa lozinke). Ovaj mikroservis takođe koristi relacionu bazu podataka. Podaci o korisniku imaju unapred definisanu strukturu, a nad korisničkim imenom i adresom postoji ograničenje da moraju biti unikatni.

- **Mikroservis za preference (eng. *UserPreferencesService*)** - Mikroservis koji je odgovoran za čuvanje svih preferenci korisnika. Čuva informacije o omiljenim slikama, žanrovima i umetnicima korisnika. Za razliku od prethodna dva mikroservisa, on koristi nerelacionu bazu podataka *MongoDB*. Pošto se sve preference korisnika čuvaju na nivou listi, kao što su sve omiljene slike korisnika, odabran je ovaj oblik baze podataka.
- **Mikroservis za narednu sliku (eng. *NextImageService*)** - Mikroservis koji generiše novu sliku za korisnika svaki dan. Uz pomoć informacija o korisniku iz mikroservisa za preference, on algoritmom odlučuje koju će novu sliku dodeliti korisniku taj dan. On čuva informaciju koje je preferentno vreme za davanje nove slike korisniku, i preko trajne veze sa korisnikom (eng. *Websocket*), šalje najnoviju sliku. Pošto i on čuva listu slika, kao i vreme kada korisnik želi novu sliku, odabrana je nerelaciona baza podataka *MongoDB*.

Svaki servis koristi alat (eng. *Maven*) koji služi za izgradnju, nad razvojnim kompletom Java 21.

3.3 Onion arhitektura unutar servisa

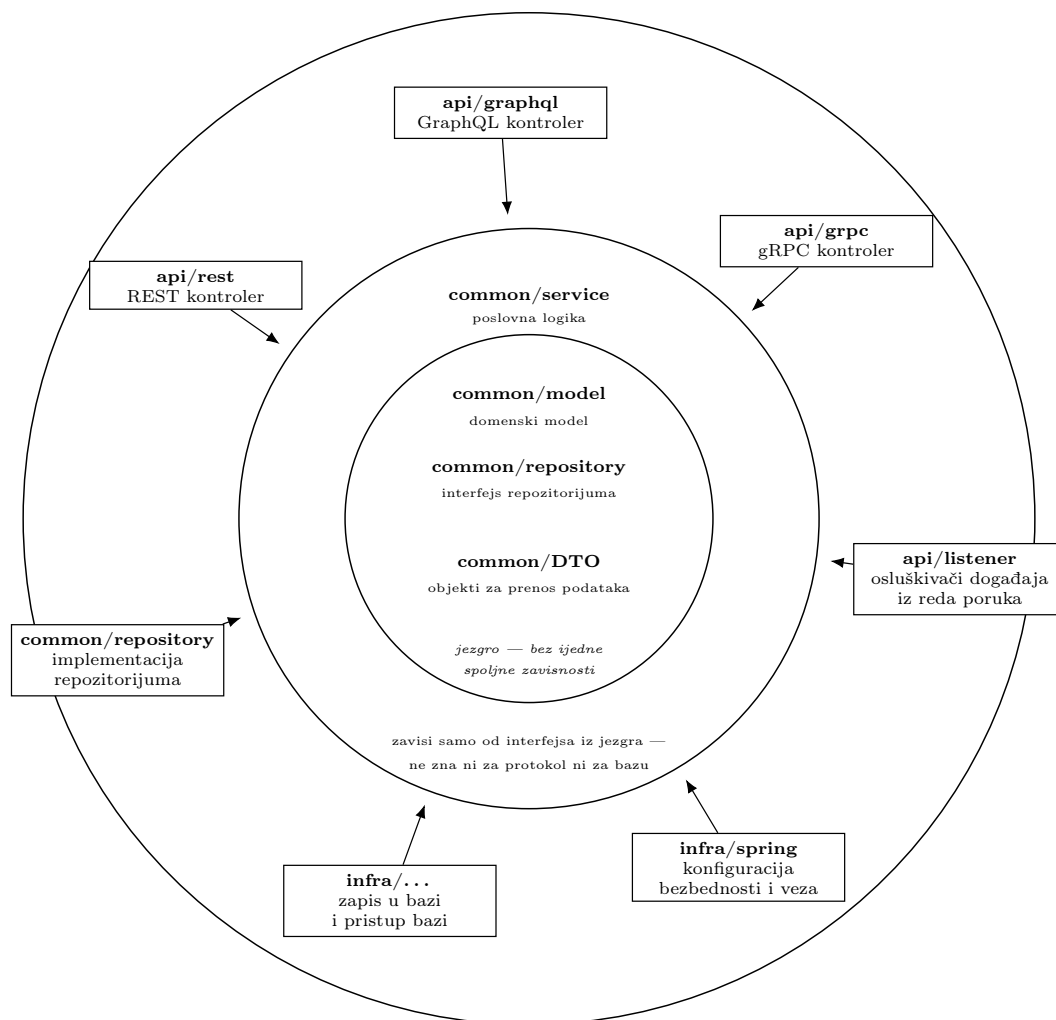
Kod unutar svakog od mikroservisa je organizovan prema (eng. *Onion*) arhitekturi. Osnovni koncept te arhitekture je da se **zavisnosti pokazuju samo ka unutra**. Svaki spoljašnji sloj zna, i može da koristi unutrašnje slojeve, dok unutrašnji slojevi ne mogu pristupiti spoljašnjim slojevima.

Na slici 3.2 se može videti kako izgleda arhitektura svakog mikroservisa. Svaki od slojeva gleda ka unutra, i upravo iz tih razloga je dodavanje više protokola komunikacije bilo olakšano. Zbog nje, svaki novi dodati protokol komunikacije će koristiti slojeve unutra, i tako će REST, gRPC, i GraphQL iako imaju različite

načine za predstavljanje programskog interfejsa, imati identičnu logiku koja se odigrava.

U jezgru se nalaze domenski modeli, objekti za prenos podataka i interfejsi repozitorijuma - dakle sve ono što opisuje *šta* aplikacija radi, a ne kako to izvodi. Oko njega stoji sloj poslovne logike, dok se u spoljašnjem sloju nalaze adapteri: kontroleri za svaki od protokola, pristup bazi i konfiguracija.

spoljašnji sloj — adapteri; jedino mesto koje zna za protokol i za bazu



Slika 3.2: Onion organizacija paketa unutar mikroservisa

3.3.1 Jezgro: entiteti i objekti za prenos

Unutar jezgra se nalaze dve različite vrste klasa. **Entiteti** predstavljaju objekte u bazi, i imaju sve informacije iz njih, dok **objekti za prenos podataka** su objekti čija je svrha komunikacija sa spoljašnjim svetom. Objekti za prenos podataka najčešće imaju manju količinu podataka, jer često nema potrebe slati sve informacije iz entiteta direktno klijentu. U kodovskom primeru 3.1 prikazan je entitet za sliku (eng. *Artwork*).

```
1 @Entity
2 @Builder
3 @Getter
4 @Setter
5 @NoArgsConstructor
6 @AllArgsConstructor
7 public class Artwork {
8
9     @Id
10    private Long id;
11    private String title;
12    @Column(length = 10000)
13    private String description;
14    @Column(name = "path_to_artwork")
15    private String imageUrl;
16
17    @ManyToOne
18    @JoinColumn(name = "artist_id")
19    @Setter
20    private Artist artist;
21
22    @ManyToMany(fetch = FetchType.EAGER)
23    @JoinTable(
24        name = "artwork_genre",
25        joinColumns = @JoinColumn(name = "artwork_id"),
26        inverseJoinColumns = @JoinColumn(name = "genre_id")
27    )
28    private Set<Genre> genres;
29 }
```

Primer koda 3.1: Primer entiteta za sliku

Za pristup relacionoj bazi korišćen je *Hibernate* [10, 11], alat za objektno-

relaciono preslikavanje (engl. *Object-Relational Mapping*, ORM). Njegova odgovornost je da premosti razliku između dva načina predstavljanja podataka: objektnog, u kome su podaci povezani referencama, i relacionog, u kome su razbijeni po tabelama i povezani stranim ključevima.

Kao posledica korišćenja *Spring Boot* radnog okvira kao i *Hibernate* alata za objektno-relaciono preslikavanje, korišćene su anotacije koje olakšavaju korišćenje entiteta. Anotacija `@Entity` označava da se klasa preslikava u tabelu relacione baze, čime je Hibernate prepoznaje kao entitet i preuzima brigu o njenom čuvanju i učitavanju. Anotacija `@Id` označava polje koje predstavlja primarni ključ te tabele, po kome se svaki red jednoznačno razlikuje od ostalih.

3.3.2 Srednji sloj: poslovna logika

Srednji sloj sadrži servise u kojima se nalazi sva poslovna logika aplikacije. On komunicira sa jezgrom, i korišćenjem entiteta iz jezgra kreira objekte za prenos, koji kasniji slojevi mogu da koriste. Na primeru 3.2 može se videti logika servisa. Njegova poslovna logika zahteva da dohvata entitete, i transformiše ih u objekte za prenos. Zbog prirode projekta, ovaj servis ne zahteva kompleksniju logiku, dok na primer, servis koji se bavi izborom sledeće slike, čiji isečak vidimo u 3.3, mora da radi kompleksnije algoritme.

```
1 @Service
2 @AllArgsConstructor
3 public class ArtworkService {
4
5     private final ArtworkRepository artworkRepository;
6     private final ModelMapper modelMapper;
7
8     public List<ArtworkDTO> getAllArtworks() {
9         return artworkRepository.findAll()
10             .stream().map(artwork -> modelMapper.map(artwork,
11                 ArtworkDTO.class))
12             .collect(Collectors.toList());
13     }
14
15     public List<IdentityArtworkDTO> getAllArtworks(String genreId) {
16         return artworkRepository.findByGenres_Id(genreId)
17             .stream().map(artwork -> modelMapper.map(artwork,
18                 IdentityArtworkDTO.class))
```

```
17         .collect(Collectors.toList());
18     }
19
20
21     public List<ArtworkDTO> getAllArtworksFromIds(List<Long>
22         artworkIds) {
23         return artworkRepository.findAllById(artworkIds)
24             .stream()
25             .map(artwork -> modelMapper.map(artwork, ArtworkDTO.
26                 class))
27             .collect(Collectors.toList());
28     }
29
30     public Optional<Long> getRandomArtworkIdExcluding(Set<Long>
31         excludeIds) {
32         if (excludeIds == null || excludeIds.isEmpty()) {
33             return artworkRepository.findRandomId();
34         }
35         return artworkRepository.findRandomIdExcluding(excludeIds);
36     }
37
38     public List<ArtworkDTO> getRandomArtworks(int count) {
39         return getAllArtworksFromIds(artworkRepository.findRandomIds
40             (count));
41     }
42 }
```

Primer koda 3.2: Primer servisa za sledeću sliku sa potrebnom poslovnom logikom

```
1 public long selectNextArtworkId(String username, Set<Long>
2     seenArtworkIds) {
3     var preferences = userPreferenceServiceClient.
4         getUserPreferences(username);
5     boolean hasGenres = !preferences.getLikedGenreIds().isEmpty
6         ();
7     boolean hasArtists = !preferences.getLikedArtistIds().
8         isEmpty();
9
10    if ((!hasGenres && !hasArtists) || random.nextDouble() <
11        probabilityConfig.getRandomImageProbability()) {
12        return getRandomUnseen(seenArtworkIds, preferences);
13    }
14 }
```

```
9
10     boolean tryGenreFirst = hasGenres && (!hasArtists || random.
        nextDouble() < probabilityConfig.
        getGenreVsArtistProbability());
11
12     Optional<Long> result = tryGenreFirst
13         ? tryFromGenre(preferences, seenArtworkIds)
14         : tryFromArtist(preferences, seenArtworkIds);
15
16     return result.orElseGet(() -> getRandomUnseen(seenArtworkIds
        , preferences));
17 }
```

Primer koda 3.3: Isečak logike izbora nove slike

Ovaj sloj nema kontekst oko protokola komunikacije koji se koristi, i upravo iz tog razloga, moguće je pozivati servis, i dohvaćanje istih podataka, bez obzira na izbor protokola.

3.3.3 Spoljašnji sloj: adapteri protokola

Spoljašnji sloj podrazumevaju klase koje rade direktnu komunikaciju sa srednjim slojem. One komuniciraju sa servisom, dohvataju podatke na koje je servis već primenio poslovnu logiku, i samo prosleđuju klijentu. U aplikaciji, primer takve klase podrazumevaju kontroleri (eng. *controller*). On definiše koji protokol komunikacije podržava. Na primeru 3.4 može se videti kontroler koji je zadužen za REST.

```
1 public class ImageController {
2
3
4     private final ArtworkService artworkService;
5     private final ArtistService artistService;
6     private final GenreService genreService;
7
8
9     @GetMapping("/get-all-artworks")
10    public List<ArtworkDTO> getAllArtworks() {
11        return artworkService.getAllArtworks();
12    }
13 }
```

```
14     @GetMapping("/get-artworks-from-genre")
15     public List<IdentityArtworkDTO> getAllArtworksFromGenre(
16         @RequestParam String genreId) {
17         return artworkService.getAllArtworks(genreId);
18     }
```

Primer koda 3.4: Isečak logike kontrolera za slike koji podržava REST

Kontroler definiše REST putanju koju podržava, i kada klijent pokuša da pristupi njoj, izvršava se određeni metod. Ovo je primer jednog protokola komunikacije koji je podržan, ali u Glavi 5, biće prikazana podrška i za ostale.

3.4 Asihrona komunikacija

Do sada sva komunikacija koja je pomenuta bila je *sinhrone prirode*, tj. pozivalac šalje zahtev i čeka odgovor. Tek kada mu stigne odgovor on nastavi dalje sa izvršavanjem, ali, ako bismo hteli komunikaciju između više mikroservisa, a da jedan mikroservis ne zna na koga sve utiče, potrebna nam je *asinhrona komunikacija*. Pošiljalac ne šalje poruku primaocu, već je šalje posredniku, i odmah nastavlja dalje sa svojim poslom. Posrednik čuva poruku, sve dok primalac nije spreman da je preuzme i da reaguje na tu poruku. Pošiljalac nema informaciju o tome ko sluša i reaguje na poruke, pa je ovakav način komunikacije prikladan za obaveštenje o događajima na koje verovatno bi neki drugi mikroservis reagovao.

Kao broker poruka (eng. *broker software*) koristi se *RabbitMQ* [12], red poruka koji implementira standard **AMQP** (engl. *Advanced Message Queuing Protocol*) [13], protokol za razmenu poruka između mikroservisa. Pošto standard definiše tačan oblik bajtova koji putuju mrežom, to omogućava da mikroservisi koji komuniciraju na ovaj način ne moraju da budu implementirani na istom programskom jeziku, i posrednik može da se zameni, bez uticaja na kod.

Primer 3.5 pokazuje isečke iz koda gde se šalje poruka, i gde se reaguje na istu. Mikroservis koji je zadužen za autentifikaciju u momentu kada se korisnik registruje šalje događaj (eng. *event*) po imenu *UserCreatedEvent*. Mikroservis zadužen za preference reaguje na taj događaj, i kreira entitet za tog korisnika.

```
1 public class UserCreatedEvent {
2     private String username;
3     private String timeZoneId;
```



```
4  }
5
6  // IdentityService -- objavljuje i nastavlja dalje
7  rabbitTemplate.convertAndSend(userEventsExchange,
    userCreatedRoutingKey, userCreatedEvent);
8
9  // UserPreferenceService -- reaguje kada mu odgovara
10 @RabbitListener(queues = "${spring.rabbitmq.user_created_queue}" )
11 public void receiveUserCreatedEvent(UserCreatedEvent
    userCreatedEvent) {
12     userPreferenceRepository.persist(userCreatedEvent.getUsername
        ());
13 }
```

Primer koda 3.5: Objava i prijem događaja o završenom registrovanju korisnika

Bitno je napomenuti da na isti događaj može više slušalaca da reaguje. Kao što mikroservis za preference reaguje na poruku da je registrovan korisnik, mikroservis za izbor sledeće slike isto sluša tu poruku, i reaguje da kreira entitet za tog korisnika.

Glava 4

Klijentska aplikacija

Klijent je realizovan u tehnologiji (eng. *React Native*) [14] uz radni okvir (eng. *Expo*) [15]. Svaka od komponenti klijenta je implementirana po model-prikaz-kontroler (eng. *Model-View-Controller*) arhitekturi [7]. U narednoj glavi biće prikazano kako su izgrađene komponente kao i dizajn obrazac (eng. *design pattern*) kad je u pitanju komunikacija sa serverom preko različitih protokola komunikacije.

4.1 React Native

Čitav klijent razvijen je u programskom jeziku TypeScript [16], uz radni okvir (eng. *framework*) *React Native* [14]. *React Native* je nadogradnja nad bibliotekom *React* [17], uz motivaciju da se iz istog koda isporuče i Android i iOS aplikacije, pri čemu se interfejs ne iscrtava u pregledaču, nego koristi izvorne komponente tih platformi. Pored toga, uz njega je korišćen i *Expo* [15], koji izmenu u kodu čini vidljivom u aplikaciji za nekoliko sekundi - što je bilo značajno jer je komunikacioni sloj tokom merenja menjan više puta - i iz tih svih razloga, klijent je izrađen u njemu.

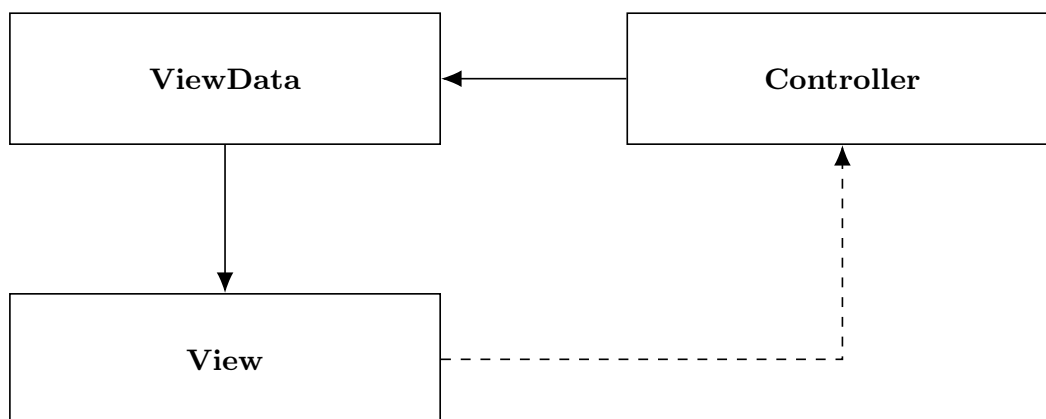
4.2 Model-Prikaz-Kontroler arhitektura

Model-prikaz-kontroler arhitektura (eng. *Model-View-Controller*) [7, 18] predstavlja ustaljen obrazac za organizaciju komponenti u klijentskim aplikacijama. Osnovne komponente arhitekture su:

- **Prikaz** (eng. *View*)

- Model (eng. *Model*)
- Kontroler (eng. *Controller*)

Na slici 4.1 vidi se upravo opisana arhitektura.



Slika 4.1: Dijagram arhitekture MVC

4.2.1 Prikaz

Prikaz je zadužen za sve komponente koje se iscrtavaju na ekranu telefona. On poseduje svoje lične podatke (u projektu pod nazivom *View Data*) koje mu se dodeljuju, i uz pomoć istih, stara se oko prikaza konkretnog dela aplikacije. U trenutku dodele novog modela od strane kontrolera prikaza, čitav prikaz se opet iscrtava. Želja ovim pristupom jeste da sve što je prikazano na ekranu uvek bude ispravno stanje koje je server poslao.

4.2.2 Model

U aplikaciji prikazni modeli (*ViewData*) predstavljaju podatke koje kontroler dodeljuje svom prikazu. Oni se razlikuju od entiteta koji su poslani od strane servera. Ideja je da model ima samo informacije koje su potrebne prikazu da vrši svoju dužnost. Model nema informaciju o samom prikazu kao ni kontroleru. On je samo klasa koja predstavlja prenos podataka.

4.2.3 Kontroler

Kontrolerova funkcija je da se bavi poslovnom logikom vezanom za prikaz. On sadrži referencu na svoj prikaz, i sa njom, dodeljuje model koji mu treba za iscertavanje ekrana. Kontroler je taj koji ima reference na repozitorijume, o kojima ćemo više videti u glavi 4.6, iz kojih čita entitete poslate od servera. U kodovskom primeru 4.1 možemo videti isečak kontrolera za početni ekran. On dohvata informacije o slikama koje se pokazuju korisniku na glavnom ekranu, vrši obradu tih podataka, i dodeljuje je svom prikazu. Metod *setViewData* je taj koji dodeljuje model prikazu.

```
1  private async load(): Promise<void> {
2      if (this.loading) return;
3      this.loading = true;
4      try {
5          const artworks = await this.buildArtworks();
6          this.setViewData(new HomeScreenViewData(artworks, true))
7              ;
8      } catch (e) {
9          console.error('[HomeScreen] loadArtworks failed:', e);
10         this.setViewData(new HomeScreenViewData(this.getSnapshot()
11             .artworks, true));
12     } finally {
13         this.loading = false;
14     }
15 }
16
17 private async buildArtworks(): Promise<FeaturedArtworkViewData
18     []> {
19     const history = await this.historyRepository.get() ?? [];
20     if (history.length === 0) return [];
21
22     const allArtworks = await this.artworkRepository.get();
23     const preferences = await this.preferencesRepository.get();
24     const likedIds = new Set(preferences?.likedArtworkIds ?? [])
25         ;
26
27     return [...history]
28         .sort((a, b) => a.seenAt.getTime() - b.seenAt.getTime())
29         .map(seenImage => {
30             const artwork = allArtworks?.getById(seenImage.
```

```
        artworkId);
27         return artwork ? new FeaturedArtworkViewData(artwork
            , seenImage, likedIds.has(artwork.id)) : null;
28     })
29     .filter((item): item is FeaturedArtworkViewData => item
        !== null);
30 }
```

Primer koda 4.1: Isečak koda iz kontrolera koji se bavi postavljanjem početnog ekrana

4.3 Namera

Prikaz nema informaciju o postojanju kontrolera, jedini način da prikaz komunicira sa spoljašnjim svetom jeste preko namera (eng. *Intent*). Namera podrazumeva neku akciju koja se dogodila na nivou prikaza, i treba ta informacija da se propagira dalje. U primeru 4.2 možemo videti ceo životni ciklus namere. U tom primeru, klasa *ArtworkPreferenceIntent* predstavlja nameru da se korisniku dopada ili ne dopada slika. Ta namera se šalje od prikaza, i stiže do kontrolera, koji dalje šalje zahtev serveru.

```
1  export type ArtworkPreferenceIntent =
2      | {type: 'LIKE'; artworkId: number}
3      | {type: 'UNLIKE'; artworkId: number}
4      | {type: 'DISLIKE'; artworkId: number}
5      | {type: 'UNDISLIKE'; artworkId: number};
6
7  // prikaz šalje nameru
8  const onPreferenceIntent = useCallback(
9      (intent: ArtworkPreferenceIntent) => send(intent),
10     [send],
11 );
12
13 // reakcija kontrolera na nameru
14 onMessage(intent: ArtworkPreferenceIntent): void {
15     this.handlePreference(intent)
16     .catch(e => console.error('[HomeScreen] preference
        intent failed:', e));
17 }
```

Primer koda 4.2: Namera da korisnik promeni preferencu prema slici

4.4 Repozitorijum

Repozitorijum [18] u klijentskom kontekstu podrazumeva čuvanje trenutnog stanja aplikacije u memoriji. Unutar repozitorijuma čuvaju se entiteti (eng. *Domain Data*) koji su kreirani iz informacija poslatih sa servera. Kod 4.3 prikazuje kreiranje entiteta za slike, koji će, pri proizvoljnom odgovoru od servera, biti sačuvan upravo u repozitorijum.

```
1 export class ArtworkData {
2     constructor(
3         readonly id: number,
4         readonly title: string,
5         readonly description: string,
6         readonly imageUrl: string,
7         readonly artist: ArtistData,
8         readonly genres: GenreData[],
9     ) {}
10 }
11
12 // kreiranje entiteta iz GRPC odgovora
13 function toArtwork(a: Artwork): ArtworkData {
14     return new ArtworkData(
15         Number(a.id),
16         a.title,
17         a.description,
18         a.imageUrl,
19         new ArtistData(Number(a.artist?.id ?? 0), a.artist?.name ??
20             ''),
21         a.genres.map(g => new GenreData(g.id, g.name)),
22     );
23 }
```

Primer koda 4.3: Entitet za sliku kao i njegovo kreiranje iz GRPC odgovora

Osnova repozitorijuma predstavlja interfejs (koji u sebi ima 3 jednostavne namene): čuvanje entiteta, upisivanje novog sadržaja u sebe, kao i da neko može

da se pretplati na promenu njenog sadržaja. Upravo taj interfejs se vidi u primeru 4.4.

```
1 export interface IRepository<T> {  
2     get(): Promise<T | null>;  
3     update(data: T): Promise<void>;  
4     subscribe(listener: () => void): () => void;  
5 }
```

Primer koda 4.4: Interfejs repozitorijuma

Sve tri metode su asinhronne prirode, iako za jednostavno čuvanje u memoriji to nije potrebno. Razlog je jer u određenim situacijama, potrebno je takođe čuvanje informacija između različitih korisničkih sesija, a čuvanje u skladište uređaja je asinhrona akcija. Kada se interfejs koristi u kodu, ne zna se koji je tačno tip repozitorijuma, što olakšava pisanje novog koda.

Postoje tri različita tipa repozitorijuma:

- **U memoriji repozitorijum** (eng. *InMemoryRepository*) - Repozitorijum koji čuva podatke u memoriji i koristi se za sve što važi samo tokom jedne sesije, poput liste slika ili istorije viđenih slika.
- **Sigurni repozitorijum** (eng. *SecureRepository*) - On čuva žetone u zaštićenom skladištu uređaja, gde ostaju i u novoj sesiji korisnika.
- **Keširani repozitorijum** (eng. *CachedRepository*) - Repozitorijum koji ne čuva ništa sam, već obavlja drugi repozitorijum i pamti pročitane vrednosti u memoriji.

4.5 Rukovalac komandama

Između klijenta i servera postoji sloj koji ih spaja, takozvani rukovalac komandama (eng. *Command Handlers*) [19]. Svaki od njih obavlja tačno jednu dužnost, da dohvati podatke sa servera i da ih upiše u repozitorijum, gde ih kasnije kontroler čita. Kontroler ne poziva server direktno, već koristi rukovaoca da izvrši komandu.

Razlog za taj sloj je što se ista radnja često poziva sa više mesta. Dohvatanje liste slika potrebno je i pri prvom pokretanju, i pri otvaranju korisničkog profila, i

posle izmene preferenci. Bez zasebnog rukovaoca, taj isti niz koraka bio bi identičan unutar svakog kontrolera.

```
1 export class GetAllGenresCommandHandler extends CommandHandler {
2     constructor(
3         private readonly client: IImageClient,
4         private readonly repository: IRepository<GenreData[]>,
5     ) {
6         super();
7     }
8
9     async handle(): Promise<void> {
10         await this.timed('GetAllGenres', async () => {
11             const genres = await this.client.getAllGenres();
12             await this.repository.update(genres);
13         });
14     }
15 }
```

Primer koda 4.5: Rukovalac komandom za dohvatanje žanrova

U primeru 4.5 može se videti kako izgleda primer takvog koda. Njegova dužnost je dohvatanje informacija od servera vezano za sve žanrove koje server podržava. On ne implementira ni jedan konkretan protokol komunikacije, već dobija referencu ka interfejsu, koji definiše sve moguće pozive ka serveru. Jedan od takvih interfejsa vidimo u primeru 4.6.

```
1 export interface IImageClient {
2     getAllArtworks(): Promise<ArtworkData[]>;
3     getArtworksFromIds(command: GetArtworksFromIdsCommand): Promise<
4         ArtworkData[]>;
5     getArtworksFromGenre(command: GetArtworksFromGenreCommand):
6         Promise<IdentityArtworkData[]>;
7     getArtworksFromArtist(command: GetArtworksFromArtistCommand):
8         Promise<IdentityArtworkData[]>;
9     getAllGenres(): Promise<GenreData[]>;
10    getAllArtists(): Promise<ArtistData[]>;
11    getAllGenresFromIds(command: GetAllGenresFromIdsCommand):
12        Promise<GenreData[]>;
13    getAllArtistsFromIds(command: GetAllArtistsFromIdsCommand):
14        Promise<ArtistData[]>;
15    getRandomArtworkId(command: GetRandomArtworkIdCommand): Promise<
```



```
        number>;  
11    getRandomArtworks(command: GetRandomArtworksCommand): Promise<  
        ArtworkData[]>;  
12 }
```

Primer koda 4.6: Interfejs svih poziva ka mikroservisu za slike

Tako, svaki od podržanih protokola komunikacije definiše interfejs po svom načinu komunikacije. Ovo olakšava pozive ka serveru sa strane kontrolera. Ne mora kontroler da bira koji će protokol komunikacije da koristi, već *Command Handler*.

4.6 Viđet

Za IOS uređaje takođe je implementiran i viđet (eng. *widget*) [20] koji se može dodati na početnom ekranu uređaja. On pokazuje sliku koju je korisnik dobio tog dana, zajedno sa imenom autora kao i imenom slike u pitanju.

Kad je implementacija viđeta u pitanju, bitna je napomena da on *nije deo aplikacije*, već na neki način, predstavlja zasebnu aplikaciju, sa kojom razvijena aplikacija u okviru ovog rada može da komunicira. Njegova logika je programirana u programskom jeziku Swift [21], i nema mogućnost da pozove metode napisane u TypeScript jeziku. Komunikacija između glavne aplikacije i viđeta je preko deljenog prostora (engl. *App Group*).

Aplikacija preuzme sliku koju treba prikazati korisniku tog dana, sačuva je u zajednički prostor, zajedno sa informacijama ko je bio autor, naziv slike, i za koji datum je slika namenjena, i onda viđet može da prikaže najnoviju sliku korisniku.

```
1  await this.timed('PublishLatestToWidget', async () => {  
2    const history = await this.historyRepository.get() ?? [];  
3    if (history.length === 0) return;  
4  
5    const sorted = [...history].sort((a, b) => a.seenAt.getTime() -  
        b.seenAt.getTime());  
6    const latest = sorted[sorted.length - 1];  
7  
8    const artwork = (await this.artworkRepository.get())?.getById(  
        latest.artworkId);  
9    if (!artwork) return;  
10 }
```

```
11     await this.publisher.publish({
12         imageUrl: artwork.imageUrl,
13         title: artwork.title,
14         artistName: artwork.artist.name,
15         imageDateISO: latest.seenAt.toISOString(),
16     });
17 });
```

Primer koda 4.7: Slanje informacije o najnovijoj slici videtu

Sam videt ima dva stanja. Kada je slika namenjena za trenutni dan, ona je prikazana. Kada nije, prelazi na prikaz koji obaveštava korisnika da postoji nova slika spremna za njega. Odluku donosi jedno poređenje datuma:

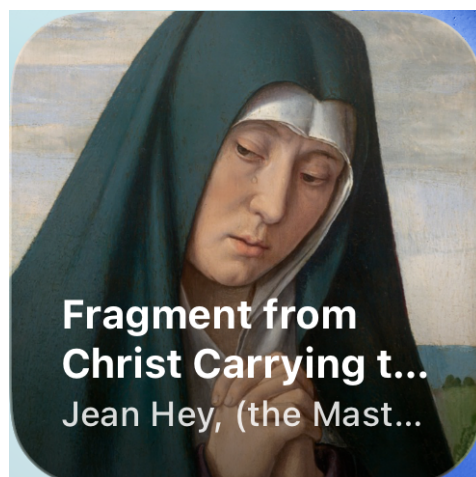
```
1 let storedDate = imageDateISO.flatMap(Self.parseISO)
2 let isToday = storedDate.map { Calendar.current.isDate($0,
    inSameDayAs: date) } ?? false
```

Primer koda 4.8: Provera da li je slika u deljenom prostoru za današnji dan

Provera u kodu 4.8 postoji da bi korisnik bio svestan da postoji nova slika spremna za njega. Ako takva provera ne bi postojala, korisnik bi, sve dok ne uđe u aplikaciju opet, video sliku koja je stara par dana. Ta stanja moguće je videti ispod na slici 4.2.



(a) nema nove slike



(b) slika za danas je spremna

Slika 4.2: Dva stanja videta na početnom ekranu

Glava 5

Protokoli komunikacije

U prethodnim glavama, sva komunikacija klijenta i servera bila je izvdvojena u zasebne slojeve. Arhitektura (engl. *Onion*) na serveru gde spoljašnji sloj predstavlja komunikaciju sa klijentom, dok na klijentu interfejsi predstavljaju izlaz ka serveru. U narednoj glavi, biće detaljnije prikazano o svakom protokolu komunikacije, kako su bili implementirani u okviru aplikacije, kao i poređenje njihovih osobina.

5.1 Računanje vremena utrošenog na čekanje

Za tumačenje rezultata merenja korisno je razložiti vreme koje korisnik čeka na sastavne delove. Ono se može izraziti kao

$$T = n \cdot RTT + \sum_{i=1}^n \left(t_i^{ser} + t_i^{obr} + \frac{S_i}{B} \right), \quad (5.1)$$

gde je n broj mrežnih kružnih putovanja, RTT vreme kružnog putovanja mreže, t_i^{ser} vreme serijalizacije i deserijalizacije, t_i^{obr} vreme obrade na serveru, S_i veličina pojedinačne poruke, a B raspoloživa propusnost.

Ova relacija pokazuje gde svaki od protokola može doneti poboljšanje. GraphQL deluje na član n , smanjujući broj kružnih putovanja time što više logičkih upita objedinjuje u jedan zahtev. gRPC deluje na članove S_i i t_i^{ser} , smanjujući veličinu poruke binarnim kodiranjem i ubrzavajući njeno pretvaranje u objekte. Nijedan od njih ne utiče na član t_i^{obr} , odnosno na vreme obrade na serveru.

Vredi naglasiti i da odnos ovih članova zavisi od okruženja. Na mobilnim mrežama vrednost RTT tipično iznosi od 30 do 100 milisekundi, pa smanjenje

broja kružnih putovanja donosi uštedu merenu stotinama milisekundi. U lokalnom okruženju, gde je *RTT* zanemarljiv, ista optimizacija gotovo da nema efekta.

5.2 REST

Termin REST (eng. *Representational State Transfer*) uveo je Roj Filding 2000. godine u svom doktorskom radu [1]. Iako je do sada spomenut kao protokol, bitna je napomena da REST nije protokol komunikacije, već *arhitektonski stil*. On predstavlja skup pravila, koja dovode do toga da sistem bude skalabilan, labavo spregnut i pogodan za keširanje.

5.2.1 Arhitektonska ograničenja

Filding navodi šest pravila (eng. *Constraints*) REST-a:

- **Klijent-Server** (eng. *Client-Server*) - razdvajanje odgovornosti između korisničkog interfejsa i skladištenja podataka.
- **Odsustvo stanja** (eng. *Statelessness*) - svaki zahtev sadrži sve podatke potrebne za njegovu obradu, a server ne čuva stanje sesije između zahteva.
- **Keširanje** (eng. *Cacheability*) - odgovori moraju biti označeni kao keširani ili nekeširani.
- **Jedinstven interfejs** (eng. *Uniform interface*) - resursi se identifikuju jedinstvenom adresom, a nad njima se vrše standardne operacije.
- **Slojevit sistem** (eng. *Layered system*) - klijent ne mora znati da li komunicira sa serverom ili preko posrednika.
- **kod na zahtev** (eng. *Code on demand*) (opciono) - server može klijentu poslati izvršivi kod.

Značajan pojam REST-a jeste **resurs**. On predstavlja svaku celinu koja je od značaja za ceo sistem. U aplikaciji, to podrazumevaju umetničke slike, žanrovi ili autori. Svaki resurs ima svoju adresu, pri čemu se razne operacije mogu vršiti nad njima:

- **GET** - dohvatanje.

- **POST** - kreiranje.
- **PUT** i **PATCH** - izmene.
- **DELETE** - brisanje.

5.2.2 Nedostaci

Glavni nedostak REST-a jeste unapred definisanje odgovora servera. Upravo zbog toga, dolazi do sledećih situacija:

- Prekomerno dohvaćanje (eng. *over-fetching*) - do ovog problema dolazi kada resurs čini više polja nego što je klijentu potrebno. Iako je klijentu potrebno manje polja nego što resurs čini, on dobija dodatne nepotrebne informacije.
- Nedovoljno dohvaćanje (eng. *under-fetching*) - predstavlja problem obrnute prirode. Pošto su klijentu potrebne informacije o svim slikama, žanrovima, umetnicima, kao i preference korisnika u isto vreme, nije moguće dobiti veću količinu podataka iz jednog dohvaćanja, već je potrebno uputiti više poziva, jer se nalaze na različitim mikroservisima. Nekada, ti resursi čak zavise jedan od drugog, pa nije moguće ni paralelizovati te pozive.

5.2.3 Implementacija

Na serverskoj strani, REST je od tri protokola najjednostavniji za uvođenje: dovoljna je anotacija nad metodom, a pretvaranje objekata u JSON obavlja sam radni okvir. Primer toga vidimo u isečku koda 5.1.

```
1 @RestController
2 @RequestMapping("/api/images")
3 public class ImageController {
4
5     private final ArtworkService artworkService;
6     private final GenreService genreService;
7
8     @GetMapping("/get-all-genres")
9     public List<IdentityGenreDTO> getAllGenres() {
10         return genreService.getAllGenres();
11     }
12
13     @GetMapping("/get-artworks-from-ids")
```

```
14     public List<ArtworkDTO> getArtworksFromIds(@RequestParam List<
        Long> artworkIds) {
15         return artworkService.getAllArtworksFromIds(artworkIds);
16     }
17 }
```

Primer koda 5.1: Isečak REST implementacije za dohvaćanje slika na serveru

Odsustvo formalnog opisa istovremeno je i najveća prednost i najveća slabost ovog pristupa: nema koda koji je potrebno generisati, ali nema ni mehanizma da definiše oblik poruke koji očekuju klijent i server. Posledica toga vidljiva je i na klijentu, u kodu 5.2.

```
1  export class RestImageClient implements IImageClient {
2
3      async getAllGenres(): Promise<GenreData[]> {
4          const response = await restFetch(
5              `${API_CONFIG.imageService}/api/images/get-all-genres`,
6              {method: 'GET'},
7          );
8          const raw = await response.json();
9          return raw.map((g: any) => new GenreData(g.id, g.name));
10     }
11 }
```

Primer koda 5.2: Isečak REST implementacije za dohvaćanje slika na klijentu

5.3 GraphQL

GraphQL predstavlja jezik upita koji unapred definiše koje sve podatke klijent može da zahteva od servera. Kreiran je od strane kompanije Fejsbuk (eng. *Facebook*) 2012. godine za internu upotrebu, ali je postao otvorenog koda 2015 godine. Osnovna zamisao protokola jeste da klijent opisuje oblik odgovora koji mu treba od servera. Umesto velike količine krajnjih tačaka, server definiše samo jednu putanju, na kojoj se može zahtevati šema (eng. *schema*), koja predstavlja strukturu koju klijent može zatražiti.

5.3.1 Schema

Schema u GraphQL podrazumeva dogovor šta je moguće klijent da dobije od servera od podataka u datom trenutku. Ako klijentu nisu potrebni svi podaci unutar scheme, već samo podskup njenih atributa, on može da zatraži upravo njih. Ovime, GraphQL izbegava problem prekomernog dohvatanja. Primer scheme za sliku može se videti u primeru 5.3.

```
1 type Artwork {
2   id: Int!
3   title: String!
4   description: String!
5   imageUrl: String!
6   artist: Artist!
7   genres: [Genre!]!
8 }
9
10 type Artist { id: Int!  name: String! }
11 type Genre  { id: ID!   name: String! }
```

Primer koda 5.3: Deo scheme mikroservisa za slike

Unutar scheme, definisana su sva polja koja server pruža za sliku. Klijent ima mogućnost da dohvati celokupnu schemu, ili samo podskup tih polja. Unutar scheme, definišu se atributi, tj. ime atributa, njegov tip i da li je striktno populisan ili ne (kada atribut ne sme biti jednak eng. *null*, definiše se sa uzvičnikom).

```
1 query PocetniEkran($ids: [Int!]!) {
2   artworksFromIds(ids: $ids) {
3     id
4     title
5     imageUrl
6     artist { name }
7   }
8   allGenres { id name }
9   allArtists { id name }
10 }
```

Primer koda 5.4: Jedan upit koji zamenjuje tri REST poziva

5.3.2 Upiti, mutacije i pretplate

GraphQL omogućava tri vrste operacija.

- Upiti (eng. **Query**) - operacija dohvaćanja i može se izvršavati paralelno.
- Mutacije (eng. **Mutation**) - operacija koja podrazumeva promenu stanja koja će biti izvršena nad podacima.
- Pretplate (eng. **Subscription**) - dugotrajna komunikacija servera sa klijentom, gde server u bilo kom trenutku može da se obrati klijentu ako postoji neki nov sadržaj.

```
1 type Query {  
2   allArtworks: [Artwork!]!  
3   artworksFromIds(ids: [Int!]!): [Artwork!]!  
4   artworksFromGenre(genreId: ID!): [IdentityArtwork!]!  
5   artworksFromArtist(artistId: Int!): [IdentityArtwork!]!  
6   allGenres: [Genre!]!  
7   allArtists: [Artist!]!  
8   genresFromIds(ids: [ID!]!): [Genre!]!  
9   artistsFromIds(ids: [Int!]!): [Artist!]!  
10  randomArtworks(count: Int! = 16): [Artwork!]!  
11  randomArtworkId(excludeIds: [Int!]): Int  
12 }
```

Primer koda 5.5: Primer upita nad slikama

Primer upita moguće je videti u primeru 5.5. Klijent ima mogućnost da u jednom pozivu, traži informacije o svim slikama, svim autorima kao i svim žanrovima odjednom. Zbog toga, nema potrebe za većim brojem poziva. Ako bi GraphQL bio definisan nad svim mikroservisima odjednom, a ne nad svakim ponaosob, bio bi rešen i problem nedovoljnog dohvaćanja, pošto bi klijent mogao sve podatke potrebne za rad aplikacije da zatraži od jednog poziva serveru.

```
1 type Mutation {  
2   addLikedArtwork(artworkId: Int!): Boolean!  
3   removeLikedArtwork(artworkId: Int!): Boolean!  
4   addLikedGenre(genreId: ID!): Boolean!  
5   removeLikedGenre(genreId: ID!): Boolean!  
6   addLikedArtist(artistId: Int!): Boolean!
```



```
7     removeLikedArtist(artistId: Int!): Boolean!  
8     addDislikedArtwork(artworkId: ID!): Boolean!  
9     removeDislikedArtwork(artworkId: ID!): Boolean!  
10 }
```

Primer koda 5.6: Primer mutacije nad preferencama korisnika

Primer koda 5.6 predstavlja mutacije koje se vrše nad preferencama korisnika. Omogućava klijentu da uputi zahtev da se desi neka promena, kao što je promena omiljene slike, uz pomoć njih.

5.3.3 $N + 1$ problem

Jedan od većih problematika GraphQL jeste $n + 1$ problem. On podrazumeva da ako klijent zahteva listu od n umetničkih slika i za svaku od njih informaciju o autoru, naivna implementacija će izvršaviti jedan upit za tih n slika i onda još n poziva za autore slika. Standardno rešenje jeste obrazac *DataLoader*, koji pozive prikuplja u okviru jednog ciklusa izvršavanja i njih izvršava kao jedan grupni upit.

Treba naglasiti da ovaj problem ne postoji samo u GraphQL, on se podjednako lako javlja i u REST implementaciji, ali ga GraphQL čini verovatnijim, budući da je oblik upita odgovornost klijenta.

5.3.4 Keširanje i cena upita

Najveća cena koju GraphQL plaća jeste gubitak keširanja na nivou protokola HTTP. Pošto se svi upiti šalju metodom POST na istu putanju, posrednici u mreži ne mogu razlikovati jedan upit od drugog, niti zaključiti da li je odgovor moguće ponovo upotrebiti. Keširanje se stoga pomera u sam klijent.

Druga cena jeste potreba za ograničenjem složenosti upita. Pošto klijent sam bira upit koji će uputiti ka serveru, taj upit može biti duboko komplikovan i da samo optereti server. Iz tog razloga, sistemi uvode ograničenje dubine.

5.3.5 Implementacija

Pre implementiranja protokola komunikacije GraphQL trebalo je odlučiti da li postaviti jedinstven ulaz (eng. *gateway*) koji objedinjuje sve mikroservise, ili svakom servisu dati sopstvenu GraphQL putanju. Izabrana je druga mogućnost iz razloga što bi jedinstven ulaz predstavljao novu komponentu koja ne postoji ni u

REST ni u gRPC postavci, čime bi poređenje postalo neravnopravno, i takođe bi uvelo dodatni mrežni skok.

Pored dodavanja scheme, kojoj se definiše međusobna komunikacija, kao i mutacije i upite, potrebni su dodaci na serveru i na klijentu za podržavanje dodatog protokola. Na nivou servera, bilo je slično kao i za REST, samo dodati novu anotaciju za svaku od metoda koji predstavljaju upit ili mutaciju. To se može videti u primeru 5.7.

```
1 @Controller
2 public class ImageGraphqlController {
3
4     private final ArtworkService artworkService;
5     private final GenreService genreService;
6
7     @QueryMapping
8     public List<IdentityGenreDTO> allGenres() {
9         return genreService.getAllGenres();
10    }
11
12    @QueryMapping
13    public List<ArtworkDTO> artworksFromIds(@Argument List<Long> ids
14        ) {
15        return artworkService.getAllArtworksFromIds(ids);
16    }
17 }
```

Primer koda 5.7: Isečak GraphQL implementacije za dohvatanje slika na serveru

```
1 const ARTWORK_FIELDS =
2     'id title description imageUrl artist { id name } genres { id
3       name }';
4
5 export class GraphqlImageClient implements IImageClient {
6
7     async getArtworksFromIds(command: GetArtworksFromIdsCommand):
8         Promise<ArtworkData[]> {
9         const data = await graphqlRequest<{artworksFromIds:
10             ArtworkDTO[]}>(<
11             ENDPOINT,
12             'query($ids: [Int!]) { artworksFromIds(ids: $ids) { ${
13                 ARTWORK_FIELDS} } }',
14             {ids: command.getIds()})
15         );
16         return data.artworksFromIds;
17     }
18 }
```

```
10         {ids: command.artworkIds},  
11     );  
12     return data.artworksFromIds.map(toArtwork);  
13 }  
14 }
```

Primer koda 5.8: Isečak GraphQL implementacije za dohvaćanje slika na klijentu

5.4 gRPC

gRPC predstavlja radni okvir za udaljeni poziv procedure (eng. *Remote Procedure Call*), razvijen od strane kompanije Gugl (eng. *Google*) i objavljen kao softver otvorenog koda 2015. godine [2]. Za razliku od REST-a, koji je zasnovan na resursima i operacijama nad njima, gRPC je zasnovan na *metodama*. Klijent ne dohvata resurs sa neke adrese, već poziva metodu koju server nudi, prosleđuje joj argumente i dobija povratnu vrednost. Zamisao jeste da poziv preko mreže izgleda što je moguće sličnije pozivu obične metode unutar same aplikacije.

Dve odluke čine osnovu ovog protokola komunikacije. Poruke se opisuju i kodiraju jezikom Protocol Buffers, a prenose se protokolom HTTP/2. Prva odluka utiče na veličinu poruke i na vreme njenog pretvaranja u objekte, dok druga utiče na način na koji zahtevi putuju kroz mrežnu vezu.

5.4.1 Protocol Buffers

Protocol Buffers (skraćeno *protobuf*) predstavlja jezik za opis strukture poruka, kao i binarni format u kom se te poruke kodiraju [22]. Opis se piše u datoteci sa nastavkom `.proto` i predstavlja dogovor kog su i klijent i server dužni da se pridržavaju. Iz tog dogovora se generiše kod na oba jezika, Java na serveru, i TypeScript na klijentu, pri čemu je onda moguća implementacija protokola. Deo opisa mikroservisa za slike moguće je videti na primeru 5.9.

```
1  syntax = "proto3";  
2  
3  package image;  
4  
5  service ImageGrpcService {  
6      rpc GetAllGenres (Empty) returns (GenreList);
```

```
7   rpc GetArtworksFromIds (IdsInt64) returns (ArtworkList);
8   rpc GetRandomArtworks (CountRequest) returns (ArtworkList);
9 }
10
11 message Artwork {
12     int64 id = 1;
13     string title = 2;
14     string description = 3;
15     string image_url = 4;
16     Artist artist = 5;
17     repeated Genre genres = 6;
18 }
19
20 message ArtworkList { repeated Artwork artworks = 1; }
```

Primer koda 5.9: Deo opisa mikroservisa za slike u jeziku Protocol Buffers

Ovime gRPC rešava problem koji je kod protokola REST ostao otvoren. Kod REST-a ne postoji formalni opis oblika poruke, pa se neslaganje između onoga što klijent očekuje i onoga što server šalje otkriva tek u vreme izvršavanja. Kod gRPC-a to neslaganje prijavljuje prevodilac, pošto su obe strane građene iz istog dogovora.

Značajan pojam jeste **redni broj polja**. Svako polje unutar poruke nosi svoj broj, i upravo se taj broj, a ne ime polja, upisuje u binarni zapis. On omogućava i razvoj scheme kroz vreme: nova polja se dodaju pod novim brojem, a stariji klijenti koja ta polja ne poznaju jednostavno ih preskaču. Iz istog razloga se jednom dodeljen broj ne sme ni promeniti ni ponovo upotrebiti za neko drugo polje.

Binarno kodiranje predstavlja glavni razlog manjih poruka. Zapis u formatu JSON za svako polje ponavlja njegovo ime, navodnike i zagrade, dok protobuf upisuje samo redni broj polja, oznaku tipa i samu vrednost. Celi brojevi se dodatno zapisuju promenljivim brojem bajtova, pa male vrednosti zauzimaju svega jedan bajt. Poređenje ova dva zapisa nad istom porukom prikazano je na slici 5.1.

5.4.2 HTTP/2 i vrste poziva

gRPC za prenos koristi protokol HTTP/2 [23], koji donosi nekoliko osobina značajnih za komunikaciju klijenta i servera:

ista poruka: *Artist { id = 1, name = "Monet" }*

<code>{"id":1,"name":"Monet"}</code>	08 01 12 05 4D 6F 6E 65 74
JSON — 23 bajta nazivi polja <code>id</code> i <code>name</code> prenose se uz svaku vrednost	Protocol Buffers — 9 bajtova prenose se samo oznake polja 1 i 2 iz opisa poruke

08 = oznaka 1, ceo broj 01 = vrednost 12 = oznaka 2, niska 05 = dužina 4D 6F 6E 65 74 = „Monet“

ušteda potiče od izostavljanja naziva polja, pa je najveća kod poruka sa mnogo malih polja, a najmanja kod poruka u kojima prevladuje dug tekst

Slika 5.1: Poređenje zapisa iste poruke u formatu JSON i u formatu Protocol Buffers

- **Multipleksiranje** (eng. *Multiplexing*) - veći broj zahteva deli jednu vezu i putuje istovremeno, bez čekanja da se prethodni zahtev završi.
- **Binarni zapis** (eng. *Binary framing*) - poruke se šalju u binarnom zapisu umesto u tekstualnom.
- **Kompresovanje zaglavlja** (eng. *Header compression*) - zaglavlja se ne prenose uz svaki zahtev, već se ponovljene vrednosti pamte.
- **Trajna veza** - ista veza koristi se za veći broj poziva, čime se izbegava ponovno uspostavljanje veze pri svakom zahtevu.

Nad tim protokolom gRPC definiše četiri vrste poziva [24]:

- **Jednostavan poziv** (eng. *Unary*) - klijent šalje jedan zahtev i dobija jedan odgovor, što odgovara klasičnom pozivu metode.
- **Tok sa strane servera** (eng. *Server streaming*) - na jedan zahtev klijenta server odgovara nizom poruka.
- **Tok sa strane klijenta** (eng. *Client streaming*) - klijent šalje niz poruka, a server odgovara tek kad je taj niz završen.
- **Dvosmerni tok** (eng. *Bidirectional streaming*) - obe strane šalju poruke nezavisno jedna od druge, kroz istu vezu.

U okviru aplikacije korišćen je isključivo jednostavan poziv, pošto nijedan slučaj upotrebe ne zahteva neprekidan tok podataka. Time se gRPC ovde koristi na isti način kao i preostala dva protokola.

5.4.3 Nedostaci

Dogovor koji postoji između klijenta i servera iako donosi prednosti, donosi sa sobom i par mana:

- **Binarni zapis poruka** - pošto su poruke koje se šalju u binarnom zapisu, nije moguće pročitati ih, bez pomoćnih alata.
- **Generisanja koda** - svaka izmena koja se napravi u *.proto* datoteci podrazumeva dodatno generisanje koda i na klijentu i na serveru.
- **Gubitak keširanja na nivou protokola HTTP** - isto kao i kod GraphQL-a, nije moguće proceniti da li se može opet iskoristiti odgovor servera.

5.4.4 Implementacija

Iz datoteke *.proto* generiše se apstraktna klasa. Nad njom se nasleđivanjem dopisuje ponašanje. Kao i kod preostala dva protokola, kontroler nema poslovnu logiku, već poziva postojeći servis. Jedina razlika jeste u tome što se odgovor ne vraća naredbom `return`, nego se predaje objektu tipa `StreamObserver`. Isti oblik metode koristi se i za pozive sa tokom podataka, pa takav ostaje i kod jednostavnog poziva. Primer se može videti u isečku koda 5.10.

```
1 @GrpcService
2 @AllArgsConstructor
3 public class ImageGrpcController extends ImageGrpcServiceGrpc.
   ImageGrpcServiceImplBase {
4
5     private final ArtworkService artworkService;
6     private final GenreService genreService;
7
8     @Override
9     public void getAllGenres(Empty request, StreamObserver<GenreList
   > obs) {
10         respondGenres(genreService.getAllGenres(), obs);
11     }
```

```
12
13     @Override
14     public void getArtworksFromIds(IdsInt64 request, StreamObserver<
        ArtworkList> obs) {
15         respondArtworks(artworkService.getAllArtworksFromIds(request
            .getIdsList()), obs);
16     }
17 }
```

Primer koda 5.10: Isečak gRPC implementacije za dohvaćanje slika na serveru

```
1 export class GrpcImageClient implements IImageClient {
2     private readonly client = createClient(ImageGrpcService,
        grpcTransport);
3
4     async getAllGenres(): Promise<GenreData[]> {
5         const res = await this.client.getAllGenres({});
6         return res.genres.map(g => new GenreData(g.id, g.name));
7     }
8
9     async getArtworksFromIds(command: GetArtworksFromIdsCommand):
        Promise<ArtworkData[]> {
10         const res = await this.client.getArtworksFromIds({ids:
            command.artworkIds.map(BigInt)});
11         return res.artworks.map(toArtwork);
12     }
13 }
```

Primer koda 5.11: Isečak gRPC implementacije za dohvaćanje slika na klijentu

Glava 6

Zaključak

Izbor protokola komunikacije između klijenta i servera predstavlja odluku koja se u razvoju mobilnih aplikacija donosi rano, i retko se menja kasnije. U ovom radu ta odluka razmotrena je na primeru aplikacije za svakodnevni prikaz umetničkih slika. Prikazan je postupak prikupljanja i obrade podataka iz javno dostupne arhive Instituta za umetnost u Čikagu, serverska strana podeljena na mikroservise od kojih svaki iznutra prati onion arhitekturu, kao i klijentska aplikacija zasnovana na obrascu model-prikaz-kontroler. Nad tako izgrađenim sistemom implementirana su sva tri protokola - REST, GraphQL i gRPC - pri čemu su svi ostali slojevi aplikacije ostali nepromenjeni. Svaki protokol predstavljen je osnovnom idejom na kojoj je izgrađen, opisom svojih ograničenja i nedostataka, kao i prikazom implementacije na serveru i na klijentu.

Sva tri protokola komunikacije implemetirana su nad istom aplikacijom, prikazane su njihove prednosti kao i nedostaci. REST se pokazao kao najjednostavnija opcija - na nivou servera potrebna je jednostavna anotacija da se doda, i radi se keširanje zahteva, koju ostali protokoli ne rade. Mana ovog pristupa je što ne postoji formalni dogovor koji se prati između klijenta i servera, što dovodi do toga da se neslaganje može primetiti tek u izvršavanju. Pored toga, postoji i problem prekomernog i nedovoljnog dohvatanja. GraphQL uklanja tu problematiku, tako što dozvoljava klijentu da definiše koje informacije o schemi su mu potrebne, i to što dozvoljava da se više zahteva spoji u jedan. Njegov nedostatak jeste keširanje zahteva, kao i problem $N + 1$. Kod gRPC-a postoji stroži oblik dogovora, jer se iz njega gradi komunikacija između klijenta i servera. Postoji dodatna brzina komunikacije, jer su poruke u binarnom zapisu. Nedostatak gRPC-a jeste upravo novo generisanje koda svaki put kad se dogovor promeni, kao i nečitljivost binarnog

zapisa.

Kao posledica iskustva dobijenog izradom ove aplikacije, nijedan od protokola komunikacije se nije pokazao kao najbolja opcija u svim slučajevima. REST kao najjednostavnija opcija kad je čitljivost odgovora servera bitna, kao i keširanje zahteva. GraphQL bi možda bio najbolja opcija za aplikaciju kao što je ova, velika količina podataka od servera u istom trenutku, gde se podaci nalaze u različitim mikroservisima. gRPC je najbolja opcija kad je zahtev da poruke budu manjih veličina, zbog binarnog zapisa, i zbog toga, deluje kao najprirodniji izbor za komunikaciju između mikroservisa na nivou servera.

Neka od ideja za nastavak istraživanja ove teme bi bilo merenje koliko je vremena trebalo svakom protokolu, gde se svaka prednost protokola iskoristi u punoj meri, kao i testiranje vremena na pravoj mobilnoj mreži. Takođe, dodavanje alata kao što su **Prometheus** i **Grafana** za grafički prikaz samih performansi. Za kraj, dodavanje aplikacije na prodavnice uređaja, gde se mogu videti performanse u pravim okolnostima.

Bibliografija

- [1] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000. on-line at: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>.
- [2] gRPC Authors. gRPC Documentation: Core Concepts, Architecture and Lifecycle, 2024. on-line at: <https://grpc.io/docs/what-is-grpc/core-concepts/>.
- [3] GraphQL Foundation. GraphQL Specification, 2021. on-line at: <https://spec.graphql.org/>.
- [4] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, 2 edition, 2021.
- [5] Jeffrey Palermo. The Onion Architecture: Part 1, 2008. on-line at: <https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/>.
- [6] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.
- [7] Martin Fowler. Model View Controller, 2002. on-line at: <https://martinfowler.com/eaCatalog/modelViewController.html>.
- [8] The Art Institute of Chicago. Art Institute of Chicago API, 2025. on-line at: <https://api.artic.edu/docs/>.
- [9] VMware Tanzu. Spring Boot Reference Documentation, 2025. on-line at: <https://docs.spring.io/spring-boot/index.html>.
- [10] Red Hat. Hibernate ORM User Guide, 2025. on-line at: <https://hibernate.org/orm/documentation/>.

- [11] Christian Bauer, Gavin King, and Gary Gregory. *Java Persistence with Hibernate*. Manning Publications, 2 edition, 2015.
- [12] Broadcom. RabbitMQ Documentation, 2025. on-line at: <https://www.rabbitmq.com/docs>.
- [13] OASIS. AMQP: Advanced Message Queuing Protocol Specification, Version 0-9-1, 2008. on-line at: <https://www.amqp.org/specification/0-9-1/amqp-org-download>.
- [14] Meta Platforms. React Native Documentation, 2025. on-line at: <https://reactnative.dev/docs/getting-started>.
- [15] Expo. Expo Documentation, 2025. on-line at: <https://docs.expo.dev/>.
- [16] Microsoft. TypeScript Documentation, 2025. on-line at: <https://www.typescriptlang.org/docs/>.
- [17] Meta Platforms. React Documentation, 2025. on-line at: <https://react.dev/>.
- [18] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [19] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [20] Apple Inc. WidgetKit Documentation, 2025. on-line at: <https://developer.apple.com/documentation/widgetkit>.
- [21] Apple Inc. The Swift Programming Language, 2025. on-line at: <https://docs.swift.org/swift-book/>.
- [22] Google. Protocol Buffers Documentation: Encoding, 2024. on-line at: <https://protobuf.dev/programming-guides/encoding/>.
- [23] Martin Thomson and Cory Benfield. RFC 9113: HTTP/2, 2022. on-line at: <https://www.rfc-editor.org/rfc/rfc9113.html>.
- [24] Kasun Indrasiri and Danesh Kuruppu. *gRPC: Up and Running*. O'Reilly Media, 2020.